A3.4 Transaction structure

This section describes the structure of transactions. The following sections define the address, data, and response structures:

- *Address structure*
- *Pseudocode description of the transfers* on page A3-48
- *Data read and write structure* on page A3-49
- *Read and write response structure* on page A3-54.

For the definitions of terms used in this section, see *Glossary* on page Glossary-301.

A3.4.1 Address structure

The AXI protocol is burst-based. The master begins each burst by driving control information and the address of the first byte in the transaction to the slave. As the burst progresses, the slave must calculate the addresses of subsequent transfers in the burst.

A burst must not cross a 4KB address boundary.

Note

This prevents a burst from crossing a boundary between two slaves. It also limits the number of address increments that a slave must support.

Burst length

The burst length is specified by:

- **ARLEN[7:0]**, for read transfers
- **AWLEN[7:0]**, for write transfers.

In this specification, **AxLEN** indicates **ARLEN** or **AWLEN**.

AXI3 supports burst lengths of 1 to 16 transfers, for all burst types.

AXI4 extends burst length support for the INCR burst type to 1 to 256 transfers. Support for all other burst types in AXI4 remains at 1 to 16 transfers.

The burst length for AXI3 is defined as,

Burst_Length = **AxLEN[3:0]** + 1

The burst length for AXI4 is defined as,

Burst_Length = **AxLEN[7:0]** + 1, to accommodate the extended burst length of the INCR burst type in AXI4.

AXI has the following rules governing the use of bursts:

- for wrapping bursts, the burst length must be 2, 4, 8, or 16
- a burst must not cross a 4KB address boundary
- early termination of bursts it not supported.

No component can terminate a burst early. However, to reduce the number of data transfers in a write burst, the master can disable further writing by deasserting all the write strobes. In this case, the master must complete the remaining transfers in the burst. In a read burst, the master can discard read data, but it must complete all transfers in the burst.

Note

Discarding read data that is not required can result in lost data when accessing a read-sensitive device such as a FIFO. When accessing such a device, a master must use a burst length that exactly matches the size of the required data transfer.

Exclusive access restrictions on page A7-93 defines additional rules affecting bursts during an exclusive access.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

In AXI4, transactions with INCR burst type and length greater than 16 can be converted to multiple smaller bursts, even if the transaction attributes indicate that the transaction is *Non-modifiable*. See *AXI4 changes to memory attribute signaling* on page A4-60. In this case, the generated bursts must retain the same transaction characteristics as the original transaction, the only exception is that:

- the burst length is reduced
- the address of the generated bursts is adapted appropriately.

Note

The ability to break longer bursts into multiple shorter bursts is required for AXI3 compatibility and might also be needed to reduce the impact of longer bursts on the QoS guarantees.

Burst size

The maximum number of bytes to transfer in each data transfer, or beat, in a burst, is specified by:

- **ARSIZE[2:0]**, for read transfers
- **AWSIZE[2:0]**, for write transfers.

In this specification, **AxSIZE** indicates **ARSIZE** or **AWSIZE**.

Table A3-2 shows the **AxSIZE** encoding.

Table A3-2 Burst size encoding

AxSIZE[2:0]	Bytes in transfer
0b000	1
0b001	2
0b010	4
0b011	8
0b100	16
0b101	32
0b110	64
0b111	128

If the AXI bus is wider than the burst size, the AXI interface must determine from the transfer address which byte lanes of the data bus to use for each transfer. See *Data read and write structure* on page A3-49.

The size of any transfer must not exceed the data bus width of either agent in the transaction.

Burst type

The AXI protocol defines three burst types:

FIXED In a fixed burst, the address is the same for every transfer in the burst.

This burst type is used for repeated accesses to the same location such as when loading or emptying a FIFO.

INCR Incrementing. In an incrementing burst, the address for each transfer in the burst is an increment of the address for the previous transfer. The increment value depends on the size of the transfer. For example, the address for each transfer in a burst with a size of four bytes is the previous address plus four.

This burst type is used for accesses to normal sequential memory.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

WRAP A wrapping burst is similar to an incrementing burst, except that the address wraps around to a lower address if an upper address limit is reached.

The following restrictions apply to wrapping bursts:

- the start address must be aligned to the size of each transfer
- the length of the burst must be 2, 4, 8, or 16 transfers.

The behavior of a wrapping burst is:

- The lowest address used by the burst is aligned to the total size of the data to be transferred, that is, to ((size of each transfer in the burst) × (number of transfers in the burst)). This address is defined as the *wrap boundary*.
- After each transfer, the address increments in the same way as for an INCR burst. However, if this incremented address is ((wrap boundary) + (total size of data to be transferred)) then the address wraps round to the wrap boundary.
- The first transfer in the burst can use an address that is higher than the wrap boundary, subject to the restrictions that apply to wrapping bursts. This means that the address wraps for any WRAP burst for which the first address is higher than the wrap boundary.

This burst type is used for cache line accesses.

The burst type is specified by:

- **ARBURST[1:0]**, for read transfers
- **AWBURST[1:0]**, for write transfers.

In this specification, **AxBURST** indicates **ARBURST** or **AWBURST**.

Table A3-3 shows the **AxBURST** signal encoding.

Table A3-3 Burst type encoding

AxBURST[1:0]	Burst type
0b00	FIXED
0b01	INCR
0b10	WRAP
0b11	Reserved

Burst address

This section provides methods for determining the address and byte lanes of transfers within a burst. The equations use the following variables:

Start_Address The start address issued by the master.

Number_Bytes The maximum number of bytes in each data transfer.

Data_Bus_Bytes The number of byte lanes in the data bus.

Aligned_Address The aligned version of the start address.

Burst_Length The total number of data transfers within a burst.

Address_N The address of transfer N in a burst. N is 1 for the first transfer in a burst.

Wrap_Boundary The lowest address within a wrapping burst.

Lower_Byte_Lane The byte lane of the lowest addressed byte of a transfer.

Upper_Byte_Lane The byte lane of the highest addressed byte of a transfer.

INT(x) The rounded-down integer value of x.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

These equations determine addresses of transfers within a burst:

- Start_Address = **AxADDR**
- Number_Bytes = 2 ^ **AxSIZE**
- Burst_Length = **AxLEN** + 1
- Aligned_Address = (INT(Start_Address / Number_Bytes)) x Number_Bytes.

This equation determines the address of the first transfer in a burst:

- Address_1 = Start_Address.

For an INCR burst, and for a WRAP burst for which the address has not wrapped, this equation determines the address of any transfer after the first transfer in a burst:

- Address_N = Aligned_Address + (N – 1) × Number_Bytes.

For a WRAP burst, the Wrap_Boundary variable defines the wrapping boundary:

- Wrap_Boundary = (INT(Start_Address / (Number_Bytes × Burst_Length)))
  × (Number_Bytes × Burst_Length).

For a WRAP burst, if Address_N = Wrap_Boundary + (Number_Bytes × Burst_Length), then:

- use this equation for the current transfer:
  - Address_N = Wrap_Boundary
- use this equation for any subsequent transfers:
  - Address_N = Start_Address + ((N – 1) × Number_Bytes) – (Number_Bytes × Burst_Length).

These equations determine which byte lanes to use for the first transfer in a burst:

- Lower_Byte_Lane = Start_Address – (INT(Start_Address / Data_Bus_Bytes)) × Data_Bus_Bytes
- Upper_Byte_Lane = Aligned_Address + (Number_Bytes – 1) –
  (INT(Start_Address / Data_Bus_Bytes)) × Data_Bus_Bytes.

These equations determine which byte lanes to use for all transfers after the first transfer in a burst:

- Lower_Byte_Lane = Address_N – (INT(Address_N / Data_Bus_Bytes)) × Data_Bus_Bytes
- Upper_Byte_Lane = Lower_Byte_Lane + Number_Bytes – 1.

Data is transferred on:

- DATA((8 × Upper_Byte_Lane) + 7 : (8 × Lower_Byte_Lane)).

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

A3.4.2 Pseudocode description of the transfers

```
// DataTransfer()
// ==============

DataTransfer(Start_Address, Number_Bytes, Burst_Length, Data_Bus_Bytes, Mode, IsWrite)
// Data_Bus_Bytes is the number of 8-bit byte lanes in the bus
// Mode is the AXI transfer mode
// IsWrite is TRUE for a write, and FALSE for a read

    assert Mode IN {FIXED, WRAP, INCR};
    addr = Start_Address;                          // Variable for current address
    Aligned_Address = (INT(addr/Number_Bytes) * Number_Bytes);
    aligned = (Aligned_Address == addr);           // Check whether addr is aligned to nbytes
    dtsize = Number_Bytes * Burst_Length;          // Maximum total data transaction size

    if mode == WRAP then
        Lower_Wrap_Boundary = (INT(addr/dtsize) * dtsize);
        // addr must be aligned for a wrapping burst
        Upper_Wrap_Boundary = Lower_Wrap_Boundary + dtsize;

    for n = 1 to Burst_Length
        Lower_Byte_Lane = addr - (INT(addr/Data_Bus_Bytes)) * Data_Bus_Bytes;
        if aligned then
            Upper_Byte_Lane = Lower_Byte_Lane + Number_Bytes - 1
        else
            Upper_Byte_Lane = Aligned_Address + Number_Bytes - 1
                              - (INT(addr/Data_Bus_Bytes)) * Data_Bus_Bytes;

        // Peform data transfer
        if IsWrite then
            dwrite(addr, low_byte, high_byte)
        else
            dread(addr, low_byte, high_byte);

        // Increment address if necessary
        if mode != FIXED then
            if aligned then
                addr = addr + Number_Bytes;
                if mode == WRAP then
                    // WRAP mode is always aligned
                    if addr >= Upper_Wrap_Boundary then addr = Lower_Wrap_Boundary;
            else
                addr = addr + Number_Bytes;
                aligned = TRUE;            // All transfers after the first are aligned
    return;
```


Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

A3.4.3 Data read and write structure

This section describes the transfers of varying sizes on the AXI read and write data buses and how the interface performs mixed-endian and unaligned transfers. It contains the following sections:

- *Write strobes*
- *Narrow transfers*
- *Byte invariance* on page A3-50
- *Unaligned transfers* on page A3-51.

Write strobes

The **WSTRB[n:0]** signals when HIGH, specify the byte lanes of the data bus that contain valid information. There is one write strobe for each eight bits of the write data bus, therefore **WSTRB[n]** corresponds to **WDATA[(8n)+7: (8n)]**.

A master must ensure that the write strobes are HIGH only for byte lanes that contain valid data.

When **WVALID** is LOW, the write strobes can take any value, although this specification recommends that they are either driven LOW or held at their previous value.

Narrow transfers

When a master generates a transfer that is narrower than its data bus, the address and control information determine which byte lanes the transfer uses:

- in incrementing or wrapping bursts, different byte lanes are used on each beat of the burst
- in a fixed burst, the same byte lanes are used on each beat.

Figure A3-8 and Figure A3-9 on page A3-50 give two examples of byte lanes use. The shaded cells indicate bytes that are not transferred.

In Figure A3-8

- the burst has five transfers
- the starting address is 0
- each transfer is eight bits
- the transfers are on a 32-bit bus
- the burst type is INCR.

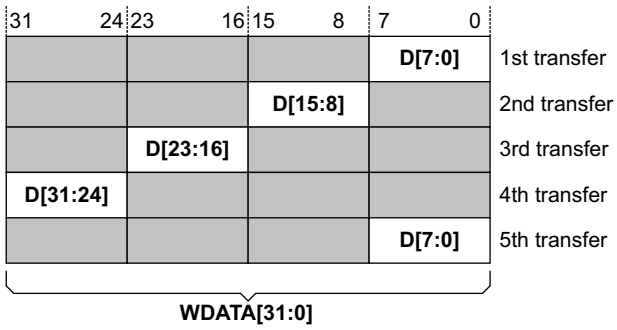


Figure A3-8 Narrow transfer example with 8-bit transfers

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

In Figure A3-9
- the burst has three transfers
- the starting address is 4
- each transfer is 32 bits
- the transfers are on a 64-bit bus.

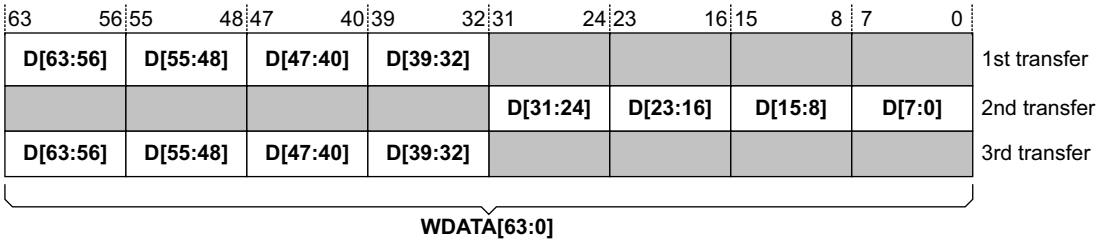


Figure A3-9 Narrow transfer example with 32-bit transfers

Byte invariance

To access mixed-endian data structures in a single memory space, the AXI protocol uses a *byte-invariant endianness* scheme.

Byte-invariant endianness means that, for any multi-byte element in a data structure:

- the element uses the same continuous bytes of memory, regardless of the endianness of the data
- the endianness determines the order of those bytes in memory, meaning it determines whether the first byte in memory is the *most significant byte* (MSB) or the *least significant byte* (LSB) of the element
- any byte transfer to a given address passes the eight bits of data on the same data bus wires to the same address location, regardless of the endianness of any data element of which the byte is a part.

Components that have only one transfer width must have their byte lanes connected to the appropriate byte lanes of the data bus. Components that support multiple transfer widths might require a more complex interface to convert an interface that is not naturally byte-invariant.

Most little-endian components can connect directly to a byte-invariant interface. Components that support only big-endian transfers require a conversion function for byte-invariant operation.

The examples in Figure A3-10 and on page A3-51 show a 32-bit number `0x0A0B0C0D`, stored in a register and in a memory.

Figure A3-10 shows an example of the big-endian, byte invariant, data structure. In this structure:
- the *most significant byte* (MSB) of the data, which is `0x0A`, is stored in the MSB position in the register
- the MSB of the data is stored in the memory location with the lowest address
- the other data bytes are positioned in decreasing order of significance.

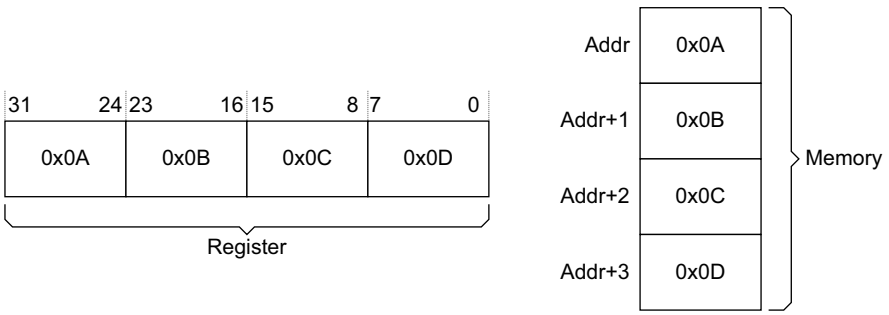


Figure A3-10 Example big-endian byte invariant data structure

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Figure A3-11 shows an example of the little-endian, byte invariant, data structure. In this structure:

- the *least significant byte* (LSB) of the data, which is `0x0D`, is stored in the LSB position in the register
- the LSB of the data is stored in the memory location with the lowest address
- the other data bytes are positioned in increasing order of significance.

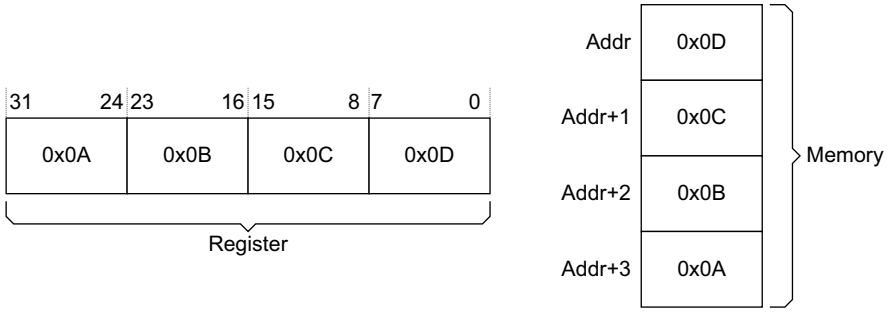


Figure A3-11 Example little-endian byte invariant data structure

The examples in Figure A3-10 on page A3-50 and Figure A3-11 show that byte invariance ensures that big-endian and little endian structures can coexist in a single memory space without corruption. Figure A3-12 shows an example of a data structure that requires byte invariant access. In this example, the header fields use little-endian ordering, and the payload uses big-endian ordering.

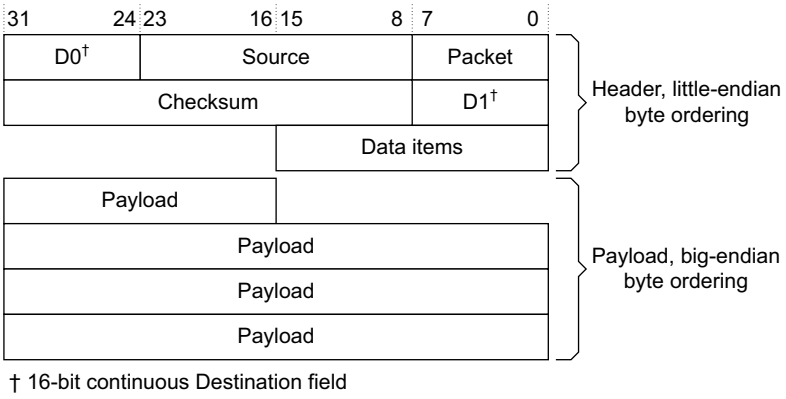


Figure A3-12 Example mixed-endian data structure

In this structure, for example, Data items is a two-byte little endian element, meaning its lowest address is its LSB. The use of byte invariance ensures that a big-endian access to the payload does not corrupt the little endian element.

Unaligned transfers

AXI supports unaligned transfers. For any burst that is made up of data transfers wider than one byte, the first bytes accessed might be unaligned with the natural address boundary. For example, a 32-bit data packet that starts at a byte address of `0x1002` is not aligned to the natural 32-bit address boundary.

A master can:

- use the low-order address lines to signal an unaligned start address
- provide an aligned address and use the byte lane strobes to signal the unaligned start address.

Note

The information on the low-order address lines must be consistent with the information on the byte lane strobes.

The slave is not required to take special action based on any alignment information from the master.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Figure A3-13 shows examples of incrementing bursts, with aligned and unaligned 32-bit transfers, on a 32-bit bus. Each row in the figure represents a transfer and the shaded cells indicate bytes that are not transferred.

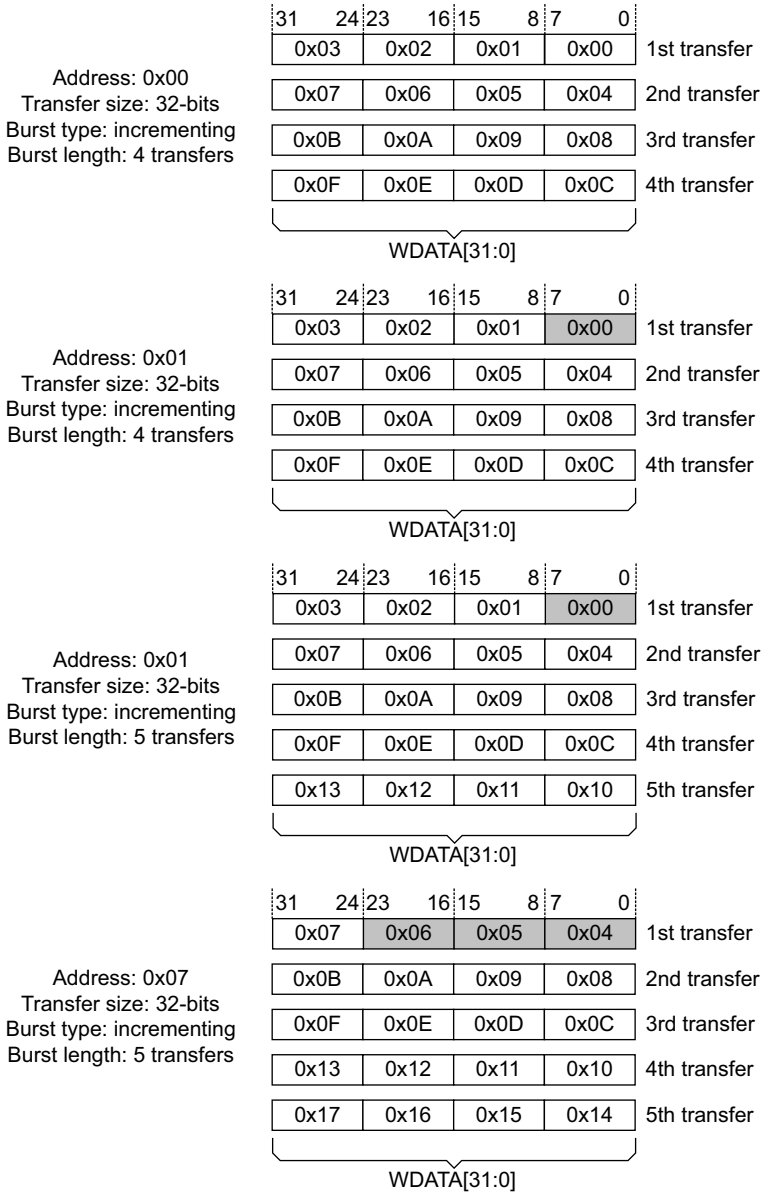


Figure A3-13 Aligned and unaligned transfers on a 32-bit bus

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Figure A3-14 shows examples of incrementing bursts, with aligned and unaligned 32-bit transfers, on a 64-bit bus. Each row in the figure represents a transfer and the shaded cells indicate bytes that are not transferred.

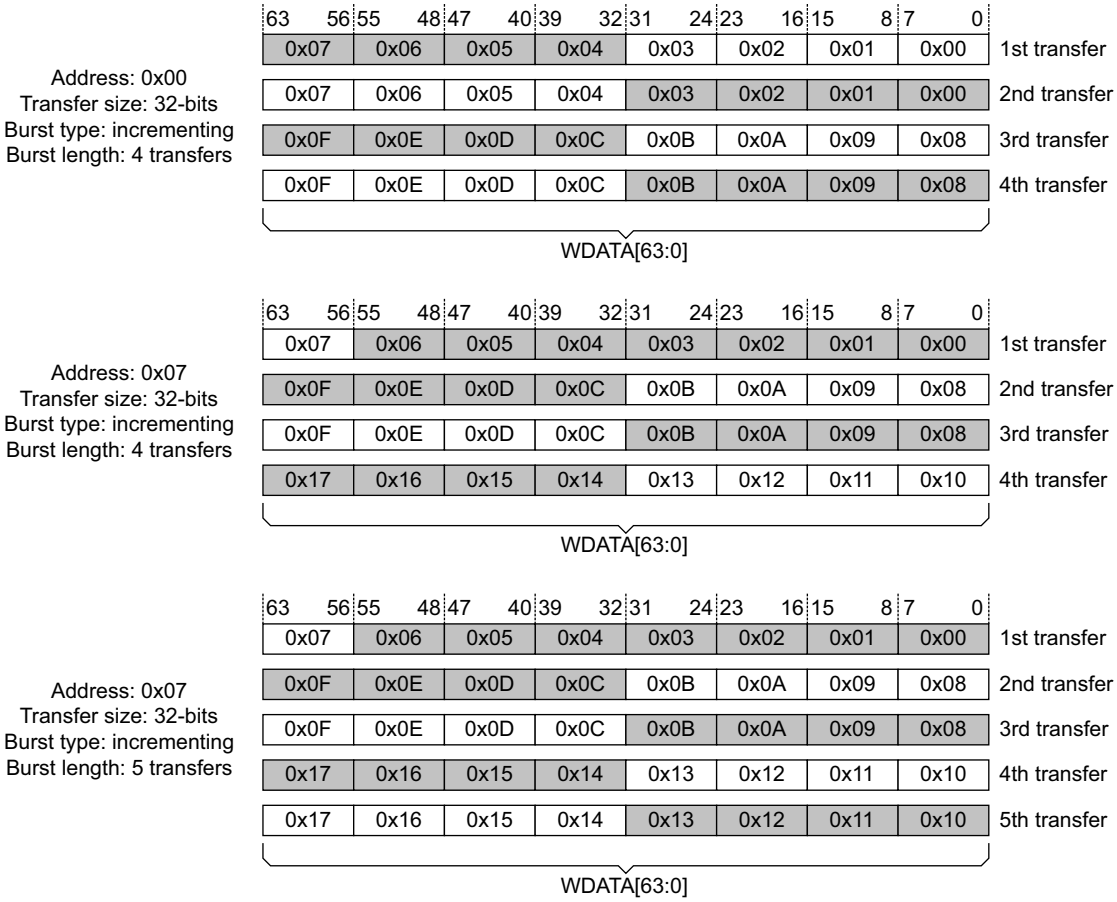


Figure A3-14 Aligned and unaligned transfers on a 64-bit bus

Figure A3-15 shows an example of a wrapping burst, with aligned 32-bit transfers, on a 64-bit bus. Each row in the figure represents a transfer and the shaded cells indicate bytes that are not transferred.

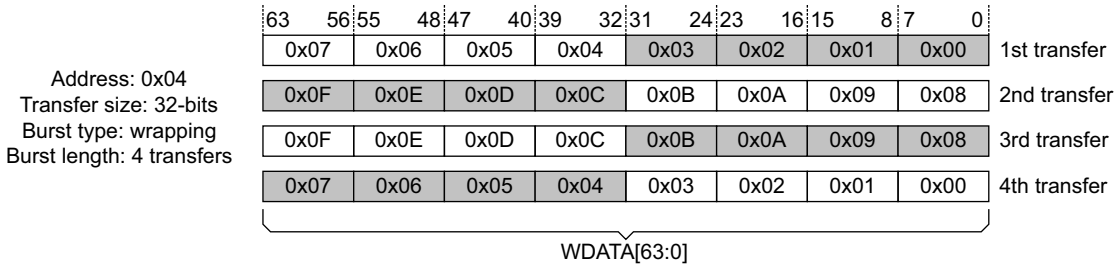


Figure A3-15 Aligned wrapping transfers on a 64-bit bus

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

A3.4.4 Read and write response structure

The AXI protocol provides response signaling for both read and write transactions:

- for read transactions the response information from the slave is signaled on the read data channel
- for write transactions the response information is signaled on the write response channel.

The responses are signaled by:

- **RRESP[1:0]**, for read transfers
- **BRESP[1:0]**, for write transfers.

The responses are:

OKAY Normal access success. Indicates that a normal access has been successful. Can also indicate an exclusive access has failed. See *OKAY, normal access success*.

EXOKAY Exclusive access okay. Indicates that either the read or write portion of an exclusive access has been successful. See *EXOKAY, exclusive access success* on page A3-55.

SLVERR Slave error. Used when the access has reached the slave successfully, but the slave wishes to return an error condition to the originating master. See *SLVERR, slave error* on page A3-55.

DECERR Decode error. Generated, typically by an interconnect component, to indicate that there is no slave at the transaction address. See *DECERR, decode error* on page A3-55.

Table A3-4 shows the encoding of the **RRESP** and **BRESP** signals.

Table A3-4 RRESP and BRESP encoding

RRESP[1:0] BRESP[1:0]	Response
0b00	OKAY
0b01	EXOKAY
0b10	SLVERR
0b11	DECERR

For a write transaction, a single response is signaled for the entire burst, and not for each data transfer within the burst.

In a read transaction, the slave can signal different responses for different transfers in a burst. For example, in a burst of 16 read transfers the slave might return an OKAY response for 15 of the transfers and a SLVERR response for one of the transfers.

The protocol specifies that the required number of data transfers must be performed, even if an error is reported. For example, if a read of 8 transfers is requested from a slave but the slave has an error condition, the slave must perform 8 data transfers, each with an error response. The remainder of the burst is not cancelled if the slave gives a single error response.

OKAY, normal access success

An OKAY response indicates any one of the following:

- the success of a normal access
- the failure of an exclusive access
- an exclusive access to a slave that does not support exclusive access.

OKAY is the response for most transactions.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

EXOKAY, exclusive access success

An EXOKAY response indicates the success of an exclusive access. See *Exclusive accesses* on page A7-92.

SLVERR, slave error

The SLVERR response indicates an unsuccessful transaction.

To simplify system monitoring and debugging, this specification recommends that error responses are used only for error conditions and not for signaling normal, expected events. Examples of slave error conditions are:

- FIFO or buffer overrun or underrun condition
- unsupported transfer size attempted
- write access attempted to read-only location
- timeout condition in the slave
- access attempted to a disabled or powered-down function.

DECERR, decode error

The DECERR response indicates the interconnect cannot successfully decode a slave access.

If the interconnect cannot successfully decode a slave access, it must return the DECERR response. This specification recommends that the interconnect routes the access to a default slave, and the default slave returns the DECERR response.

The AXI protocol requires that all data transfers for a transaction are completed, even if an error condition occurs. Any component giving a DECERR response must meet this requirement.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Chapter A4
Transaction Attributes

This chapter describes the transaction attribute signaling that supports system topology and system level caches. It contains the following sections:

- *Transaction types and attributes* on page A4-58
- *AXI3 memory attribute signaling* on page A4-59
- *AXI4 changes to memory attribute signaling* on page A4-60
- *Memory types* on page A4-65
- *Mismatched memory attributes* on page A4-69
- *Transaction buffering* on page A4-70
- *Access permissions* on page A4-71
- *Legacy considerations* on page A4-72
- *Usage examples* on page A4-73.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential